

Cost-Efficient Optimization of Parallel Radix Sort Using Architecture-Aware Techniques

Sayem Kamal
Columbia University
sk5336@columbia.edu

Gianna Belmont
Columbia University
gb2973@columbia.edu

Soaiba Nuzhat
Columbia University
sn3093@columbia.edu

Sharvani Vadlamani
Columbia University
sv2734@columbia.edu

Abstract—Radix Sort is a memory-bound algorithm whose performance is closely tied to processor microarchitecture. This work evaluates and optimizes a parallel LSD radix sort on the AMD EPYC 7302P (“Rome”) platform, focusing on architectural techniques rather than compiler-level tweaks. By combining L3-aware blocking, per-thread histograms, CCX-aware thread pinning, hugepages, and tuned multithreading, we substantially increase effective memory bandwidth and reduce DRAM pressure. The optimized implementation achieves up to a 57x speedup over the baseline at 32 threads and 17.6x at 16 threads, ultimately plateauing at the system’s 204.8 GB/s bandwidth limit while also reducing cost-per-GB by 17.2x on large datasets. Overall, the results show that hardware-aware optimizations are more effective than compute- or compiler-based approaches for bandwidth-bound workloads.

I. INTRODUCTION

Large scale, stable, in-memory sorting is a fundamental operation in data processing pipelines, warranting efficient sorting algorithms that consider hardware architectures rather than solely algorithmic complexity. In this report, we study the performance and cost-efficiency of optimizations on least significant digit (LSD) radix sort tailored to the microarchitectural characteristics of the AMD EPYC 7302P processor.

We selected LSD Radix Sort over comparison-based algorithms (e.g., Quicksort, Timsort) due to its $O(N)$ complexity and branch-free/bitwise logic, which eliminates the branch misprediction penalties of the other sorting algorithms. However, Radix Sort is memory-intensive, often incurring high TLB pressure, poor spatial locality, and cache thrashing. Hence, being conscious of hardware, we focused our optimizations on these bottlenecks, implementing a layered optimization stack. This includes CCX-aware thread pinning, hugepages, multithreading, and cache blocking, designed to virtualize the memory hierarchy and maximize locality.

All experiments were conducted on the CloudLab c6525-25g node (16-core). We explicitly selected this hardware over the 24-core variant (c6525-100g) because both share the same 204 GB/s memory bandwidth limit, and additional cores would disproportionately increase cost compared to speed, degrading cost-efficiency.

We hypothesized that a block size of 256KB would be optimal, corresponding to half of the Zen 2 architecture’s 512KB L2 cache, theoretically allowing a sorting tile to reside entirely within the fastest private cache level. Furthermore, we hypothesized that 16 threads would present the optimal operating point, where simultaneous multithreading to reach

32 threads would introduce cache pollution, degrading performance.

Using these hypothesized parameters (16 threads/256KB block size) on a dataset of one billion random integers (4GB), we achieved an initial 12.84x speedup with full compiler optimizations. However, after adjusting to parameters from the experiments outlined in our results (32 threads/8MB block size), we achieved 17.6x speedup to baseline with compiler optimizations, and 56x speedup without any compiler optimizations. Furthermore, as input size increases, we see greater speedups from the optimized version compared to the baseline.

II. METHODOLOGY

A. Hardware Overview

The hardware utilized for this project was the CloudLab Utah c6525-25g machine equipped with an AMD EPYC 7302P (“Rome”) CPU. Consisting of 16 cores (32 threads), a 3.0 GHz base clock, and 128 GB of DDR4-3200 spread across 8 memory channels, it was determined that this machine was the optimal choice for various reasons. Overall, since radix sort moves large amounts of data every pass, the available memory bandwidth of this system proves most efficient for the task at hand.

TABLE I
25G VS 100G NODE COMPONENTS

Component	25g Node	100g Node
CPU Model	AMD EPYC 7302P	AMD EPYC 7402P
Cores/Threads	16 / 32 SMT	24 / 48 SMT
Base/Boost Clock	3.0 / 3.3 GHz	2.8 / 3.35 GHz
L1 Data Cache	32 KB/core	32 KB/core
L2 Cache	512 KB/core	512 KB/core
L3 Cache	16 MB per CCX	16 MB per CCX
Memory	128 GB DDR4-3200	128 GB DDR4-3200
Theoretical BW	204.8 GB/s	204.8 GB/s
Cost/Hour (Azure)	\$0.9040/hr	\$1.3560/hr

The c6525-25g (AMD EPYC 7302P) offers a critical architectural advantage over the c6525-100g (AMD EPYC 7402P) regarding L3 cache contention. Both processors feature a total of 128 MB of L3 cache, which is physically distributed in 16 MB slices, with one slice dedicated to each Core Complex (CCX). Both processors utilize 8 active CCXs. However, the distribution of cores within these complexes differs significantly. On the 100g node, there are 3 cores per CCX. This forces 3 cores to compete for the same 16 MB cache slice,

yielding approximately 5.33 MB of L3 cache per core. In contrast, the 25g node has 2 cores per CCX. This configuration grants the 25g node a superior 8 MB of L3 cache per core. For a memory-intensive algorithm like Radix Sort, this 50% increase in available local cache significantly reduces the likelihood of cache thrashing and lowers contention on the Infinity Fabric, making the 25g node the more performant choice for per-core efficiency.

Since CloudLab lacks specific node pricing, we utilized Azure ‘pay-as-you-go’ estimates for our cost-efficiency analysis, comparing the c6525-25g to Azure’s E16as_v5 (16vCPU, 128GiB) at approximately \$0.9040/hr. For c6525-100g, since Azure does not exactly have a 24-vCPU/128GiB configuration, we use the E-series E32_asv5 (32 vCPU, 256GiB) and linearly scaled the price to 24vCPU, yielding a \$1.3560/hr price. Given this significant price difference, the 100g node would need to perform at least 1.5x faster than the 25g node to be cost-efficient; however, we hypothesize this speedup is unattainable due to Amdahl’s Law, which limits theoretical gains based on sequential bottlenecks, specifically the 204.8 GB/s memory bandwidth limit shared by both nodes. Because the 100g node’s additional cores face diminishing returns from bus contention and run at a lower base clock frequency to maintain thermal headroom, they offer slower per-core performance without alleviating the primary memory bottleneck, making the c6525-25g the superior choice for maximizing performance-per-dollar.

B. Baseline Implementation

Our baseline was a straightforward, unoptimized implementation of LSD radix sort compiled with GCC. We continued using GCC for all subsequent optimizations. Because radix sort is overwhelmingly memory-bound on EPYC Rome, we hypothesized that compiler choice would have a minimal effect on performance. Once the implementation saturates memory bandwidth, differences in instruction scheduling or vectorization between compilers (e.g., GCC, Clang, ICC) cannot meaningfully improve throughput. Thus, GCC was sufficient and appropriate for all experiments. Several properties of radix sort influenced us to implement LSD (least-significant-digit) radix sort with 8-bit digits (256 buckets). Radix sort achieves linear ($O(N)$) time complexity, which is advantageous for a multi-gigabyte dataset. Moreover, radix sort avoids branch mispredictions on random data because it is not a comparison-based approach. Finally, radix sort works well with our L1 cache; A 256-entry histogram is only 1 KB, so even thread-local histograms fit comfortably within the cache.

The sorting algorithm performs four passes, one byte at a time, and uses a simple three-step structure for each pass: zero the count[256] array, scan the input to build frequency counts, compute a prefix sum, and then scatter elements into a temporary workspace buffer. The scatter phase is the dominant cost because its irregular writes create poor cache locality, frequent TLB misses, and heavy DRAM traffic. After each pass, the workspace is copied back into the main array before moving on to the next byte.

The main function allocates the input and workspace buffers, generates a dataset (default: 1 billion elements), times the sort, and checks correctness. This version includes no architecture-specific tuning, no blocking, no parallelism, and no thread pinning, so it gives us a clear baseline for how radix sort behaves when limited mainly by memory bandwidth on our system.

C. Optimizations

1) *L3-Aware Cache Blocking*: We initially considered a 256 KB block size to target the 512 KB private L2 cache, under the assumption that a block and its histogram could remain in L2 during the counting and scattering phases. The final design instead uses multi-megabyte tiles sized to fit within a single CCX’s 16 MB L3 slice. Using 2 MB blocks keeps each thread’s working set local to one CCX, reduces cross-CCX traffic, and improves locality throughout the pass. The implementation also includes software prefetching to overlap memory latency within each tile, which slightly improves throughput by pulling future cache lines into the L3 slice before they are needed.

2) *Parallel Local Histograms*: Since the baseline implementation relies on a single global histogram, multiple threads end up updating the same counters, causing the cache lines to bounce back and forth between cores and significantly hurting performance. To eliminate this bottleneck, each thread maintains a private 256-entry histogram, and the results are merged once per pass. This avoids coherence traffic during counting and enables near-linear scaling up to the 16 physical cores, with additional gains from SMT up to 32 threads.

3) *CCX-Aware Thread Pinning*: EPYC Rome groups every four cores into a CCX, each with a private 16 MB L3 slice. If a thread migrates to a different CCX, it must refetch the data it was using into the new L3 slice, which adds delay and increases traffic on the Infinity Fabric. To avoid this, we pin each OpenMP thread to a specific core to preserve L3 locality and reduce runtime variance. While this does not raise peak bandwidth, it preserves locality and avoids unnecessary cross-CCX activity, resulting in more stable performance, especially when SMT is enabled.

4) *Hugepages*: Radix Sort makes repeated streaming passes over large arrays, which puts significant pressure on the TLB. With standard 4 KB pages, a 4 GB dataset requires over one million page mappings, while using 2 MB hugepages reduces this to roughly two thousand. Enabling transparent hugepages lowers TLB miss rates and cuts down on page-walk overhead during the scatter phase, providing modest but reliable improvements in effective bandwidth.

III. RESULTS AND ANALYSIS

A. Number of Threads vs Speedup

To determine the optimal threading model and validate our hardware selection, we measured the speedup factor of our optimized implementation compared to its single-threaded performance as thread count increased from 1 to 32. This

experiment explicitly tests Amdahl’s law and the effects of SMT.

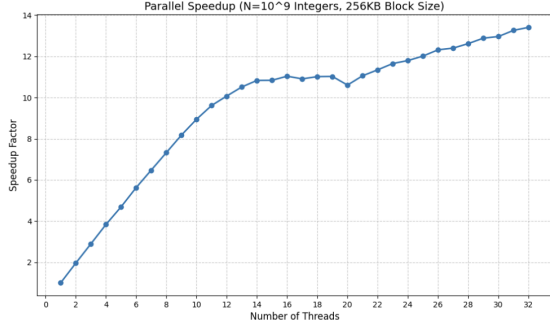


Fig. 1. Thread scaling performance, $N=10^9$ integers, block=256KB

As shown in Fig. 1, the performance curve exhibits two phases. Up to 16 threads, speedup is nearly linear (11.0x at 16 threads). Beyond 16 threads (the SMT region), the slope flattens, with 32 threads delivering 13.4x speedup. This confirms that memory bandwidth, not compute, limits scalability.

This validates our choice of the 16-core c6525-25g node: SMT provides latency hiding, but additional physical cores from the 100g node would not yield proportional gains.

B. Block Size Sweep

We next swept block sizes from 16KB to 64MB to test our hypothesis that 256KB (half of L2) would be optimal.

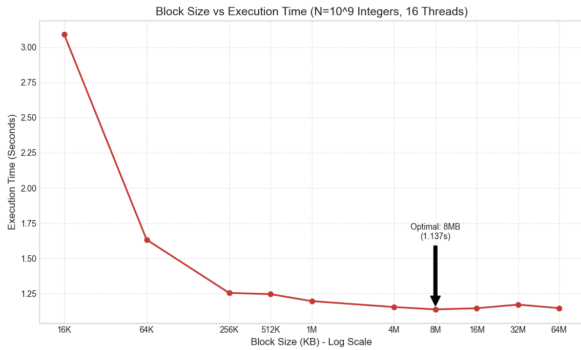


Fig. 2. Block size sweep, $N=10^9$ integers, 16 threads

Fig. 2 shows that blocks from 16KB–256KB underperform due to initialization and synchronization overhead. Performance improves steadily up to 4MB and peaks at 8MB. Surprisingly, performance remains flat beyond 8MB, indicating bandwidth saturation. This contradicts our initial L2-based hypothesis and instead shows that L3 residency (16MB CCX slices) governs optimal block size.

C. Compiler-Driven Improvements

We compared the baseline and optimized implementations at -O0, -O2, and -O3 -march=znver2.



Fig. 3. Compiler optimization impact, $N=10^9$ integers, 32 threads, 8MB block

As shown in Fig. 3, at -O0 the optimized version is 57x faster than baseline (1.41s vs 80.04s). With -O3, speedup is 17.6x. The optimized version does not improve from -O2 to -O3, confirming full memory-bandwidth saturation: additional vectorization cannot improve fetch throughput.

D. Scalability

We swept dataset sizes from 10^7 to 10^9 integers.

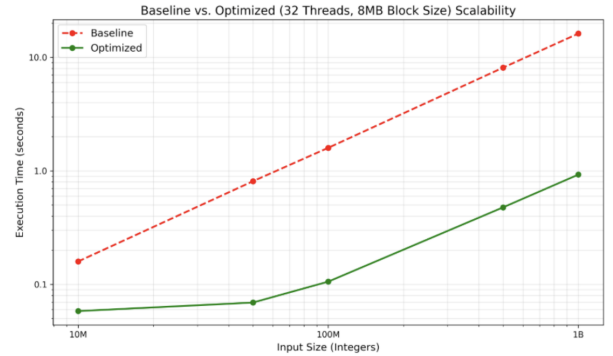


Fig. 4. Scalability across input sizes, 32 threads, 8MB block

Fig. 4 shows the baseline scaling linearly from the outset, immediately limited by DRAM. The optimized curve is sublinear up to 100M elements, indicating L3 residency of working tiles. At 1B elements, scaling slope matches the baseline, confirming the transition to a DRAM-bound regime.

E. Cost Efficiency Across Input Sizes

Using the c6525-25g Azure-equivalent rate of \$0.9040/hr, cost is computed as:

$$\text{Cost} = \text{Time (s)} \times \left(\frac{0.9040}{3600} \right)$$

$$\text{Cost per GB} = \frac{\text{Cost}}{\text{Dataset Size (GB)}}$$

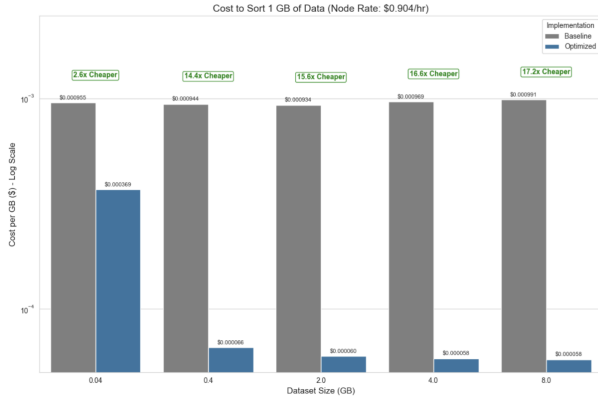


Fig. 5. Cost per GB vs dataset size, 32 threads, 8MB block

The baseline shows a flat, high cost profile. The optimized implementation, however, amortizes fixed costs as dataset size grows, resulting in a 17.2x cost reduction at the 8GB scale. This highlights the economic value of architectural optimizations on memory-bound workloads.

IV. LIMITATIONS AND CHALLENGES

While our optimizations delivered substantial speedups, several limitations constrained our results. First, radix sort on EPYC Rome is fundamentally memory-bandwidth bound, meaning that once the implementation saturates the 204.8 GB/s DRAM limit, neither additional cores nor more aggressive compiler optimizations can meaningfully improve throughput. As a result, microarchitectural tuning exhibits diminishing returns beyond L3-local optimizations.

Second, CloudLab does not expose LLC misses or TLB miss rates, limiting our ability to attribute performance gains to specific microarchitectural effects. This lack of visibility makes it challenging to precisely quantify the benefits of hugepages, prefetching, or block-size selection. Additionally, CloudLab does not provide an official pricing model for node usage, requiring us to infer cost-efficiency using approximations based on cloud providers such as Azure. These estimates, while reasonable, introduce uncertainty into our cost-per-GB analysis because they may not accurately reflect the true economic tradeoffs of running on CloudLab hardware.

Together, these constraints highlight that while our optimizations are effective, they limit the precision with which we can analyze and explain the observed performance gains.

V. CONCLUSION

Based on the experimental results, we can conclude that optimizing Radix Sort for the AMD EPYC "Rome" architecture requires a structural approach centered on saturating memory bandwidth rather than increasing raw compute power. Our thread scaling experiments validated the selection of the c6525-25g (16-core) node, achieving a peak 13.4x speedup and confirming that SMT (Simultaneous Multithreading) successfully hides memory latency without requiring the costlier 24-core alternative. Contrary to our initial hypothesis of a 256KB optimal block size, parameter sweeps revealed that

8MB blocks maximized L3 cache utility, effectively shielding cores from main memory latency until the dataset exceeded cache capacity. This architectural tuning proved far more critical than standard compiler optimization. With no compiler optimizations, our structural improvements delivered a 57x speedup. With full compiler optimization, we saw a 17.3x speedup. It was also observed that moving from -O2 to -O3 yielded zero additional gains, confirming the workload is strictly bound by the 204.8 GB/s physical memory channels. In future work, we hope to run our implementation on hardware platforms that expose detailed performance counters, such as LLC misses and TLB activity, to gain deeper insight into the microarchitectural effects driving our speedups. Access to these counters would allow us to directly verify the impact of hugepages, thread pinning, and block-size choices beyond run-time measurements. Ultimately, these optimizations translated into tangible economic efficiency, reducing the cost-per-GB by 17.2x at the 8GB scale, validating a design philosophy that prioritizes cache residency and bandwidth management over simple core count.